

学号：22920202200773

姓名：孟媛

实验一 二分法求方程的根

一、实验目的

使用二分法求方程的根。

二、数值计算原理

如果 $f(a) \cdot f(b) < 0$ ，则必有一根在 a 与 b 之间，取 a 与 b 的中点 x ，若 x 是根，则找到解，如果 $f(a) \cdot f(x) < 0$ ，则 a 与 x 之前必有一根，否则在 b 与 x 中必有一根，不断取中点迭代，就可以求出 (a,b) 中的一个根。

三、源程序介绍

函数接受三个参数：求根区间下限、求根区间上限、精度，输出求根过程，返回最终求得的足够精度的根，并能对错误输入进行提示。

求解过程如下：

- (1) 先求出边界 (l, r) 的函数值 $f(l), f(r)$ ，如果二者符号相同或左区间 $l > r$ 则错误提醒并退出程序
- (2) 如果区间大小小于给定的精度，退出
- (3) cnt 记录迭代的次数，每次+1，计算中点 $mid = (l + r)/2$ 。
- (4) 计算中点的函数值
- (5) 如果 $f(mid)$ 和 $f(l)$ 的函数值符号相反，将 mid 赋值给 r
- (6) 如果 $f(mid)$ 和 $f(l)$ 的函数值符号相同，将 mid 赋值给 l
- (7) 转到 (2)

```

1.py > ...
1  #二分法求方程的根
2  import numpy
3  from scipy import optimize
4
5  cnt = 0 #计数二分次数
6
7  def func(x):
8      return numpy.sin(x) - x * x / 4
9
10 def check(l, r, st): #参数依次为左区间,右区间,精度
11     if (func(l) * func(r) > 0 or l > r):
12         print("无根或输入异常!")
13         exit(0)
14     global cnt
15     while (numpy.fabs(r - l) >= st):
16         cnt += 1
17         mid = (l + r) / 2
18         if (func(l) * func(mid) < 0):
19             r = mid
20         else:
21             l = mid
22     return l
23
24 if __name__ == '__main__':
25     l = float(input("请输入左区间, 以回车结束:"))
26     r = float(input("请输入右区间, 以回车结束:"))
27     st = float(input("请输入精度:"))
28     print("自编函数求得方程的解为:", check(l, r, st), ", 二分进行了:", cnt, "次")
29     print("自带函数求得方程的解为:", optimize.newton(func, x0 = l, x1 = r, tol = st))

```

四、程序执行情况

4.1 书上给的例子:

```

def func(x):
    return numpy.sin(x) - x * x / 4

```

```

请输入左区间, 以回车结束:1
请输入右区间, 以回车结束:10
请输入精度:0.000000001
自编函数求得方程的解为: 1.9337537625106052 , 二分进行了: 34 次
自带函数求得方程的解为: 1.9337537628270212

```

4.2 书上未给的例子

```

def func(x):
    return numpy.exp(x) - x * x / 4

```

```
请输入左区间，以回车结束:-100  
请输入右区间，以回车结束:10  
请输入精度:0.00000001  
自编函数求得方程的解为: -1.1342865868937224 , 二分进行了: 34 次  
自带函数求得方程的解为: -1.1342865808195677
```

五、结果分析

分析发现，二者均要求根区间上下限以及精度，但二者所求得同一精度的根并不完全相同。此外自带函数 `optimize.newton` 还需要上传函数名。

相较不动点迭代等方法，二分法的迭代速度慢、迭代次数多，不适合快速求解。

六、实验体会

二分法思想较为简单，代码编写也较为简单，但是，迭代速度过慢且比较复杂，认识到了它的局限性。除此之外，起初我使用了 C++ 编写，但是无奈没能下载成功第三方库找到合适工具函数作一个对比，转而只好使用 C++ 重新编写。此次，我也更为认识到了 python 在某种意义上的优越性以及自身能力的不足。